Digital Alpha Technologies

Full Stack Engineer (Frontend-Focused) - Take-Home Assignment

Hi, and thanks for your interest in joining Digital Alpha Technologies.

This assignment is meant to reflect the kind of work you'd actually do here: taking a written brief for a data-heavy financial app and building a working slice of it end to end, frontend through database. Please read the whole thing before you start coding. It's short.

You have 24 hours from the time this email reaches you. We're not expecting a finished startup. Plan for something like a solid day of focused work, scope accordingly, and please don't lose a night of sleep over it. We'd much rather see a smaller feature set done well than everything half-built.

What you're building

A consumer app for paying credit-card bills, earning reward coins on payments, and looking at your own spending. You'll build the core of it: a transactions and rewards dashboard, backed by a real API and database. The idea is deliberately simple so the interesting part is *how* you build it, not what it is.

Call the project whatever you like. There's no fixed name and no design file. The look and feel is yours to decide, and we do pay attention to visual quality and polish, so treat the UI seriously.

Wherever the brief leaves something open (and it does in a few places), make a reasonable product call and note it down. We're genuinely interested in the assumptions you make, so keep track of them as you go.

Requirements

Transactions dashboard

The dataset attached with this email (`transactions.json`) has around 10,000 transactions. Show them in a table that stays smooth with the full set loaded. Pagination or virtualization is up to you, just be ready to explain why you picked one.

Users should be able to filter (by category, date range, amount range, and payment status, combinable), search merchant names as they type, and sort by date and amount. Clicking a row should open its full detail somewhere sensible, a drawer or a modal, your call.

Spend analytics

Give the user a view of their spending by category and a monthly trend over time. At a minimum, clicking a slice of a chart should filter the transaction table. Making it fully two-way (table filters also reshaping the charts) is a nice step up if you have time. Keep this responsive with the full dataset.

Rewards

Users earn coins on successful payments, one coin per ₹100 spent, capped per transaction. Their coin balance should always be visible somewhere. Let them redeem coins against a small catalogue of four to six rewards that you define (vouchers, cashback, whatever fits). The redeem flow is select, confirm, done. If the redeem

call fails, the UI has to recover cleanly rather than leaving the balance in a wrong state, and the backend has to reject a redeem when the balance is too low or the reward doesn't exist, with sensible status codes.

Where to focus

You won't finish everything perfectly in a day, and you're not expected to. Here's how we'd spend the time, in order. Get the earlier things solid before reaching for the later ones.

Core (do these first, do them well) - Transactions table on the full 10k rows: filter, search, and sort, staying smooth. - One spend chart (category breakdown or monthly trend). - Rewards: visible coin balance and a working redeem flow (select, confirm, balance updates), with the backend rejecting an invalid or unaffordable redeem. - PostgreSQL with a schema and a one-command seed. - Deployed, or a short demo video if you couldn't deploy in time.

Nice to have (if the core is solid) - The second chart and one-way chart-to-table filtering. - Server-side pagination, filtering, and sorting (rather than doing it all in the browser). - Optimistic balance update with a clean rollback when the redeem call fails. - A polished, hand-built modal (focus trap, Escape to close).

Bonus (genuinely optional) - Two-way cross-filtering between charts and table. - A test or two, accessibility touches, a walkthrough video even if you deployed.

A tight, well-built core beats every feature half-done. We mean that, and we score it that way.

Tech we expect

Frontend (this is where most of your time should go, roughly 70 to 75 percent):

React with TypeScript, Next.js preferred. We care a lot about how you structure components, so build a small internal system of your own: design tokens for colour, spacing and type, plus reusable pieces like Button, Card, Table and Modal.

One constraint worth reading twice: build the **Table yourself, without a component library** (no MUI, Ant, Chakra, shadcn and so on for the table). This is the main place we look at your CSS, things like a sticky header, proper hover, focus, loading, empty and error states, and a layout that holds together down to 360px wide. For everything else, including the modal or drawer, use a library if you like, though a cleanly hand-built modal (focus trap, Escape to close) is a nice signal if you have the time. Charting libraries like Recharts, Chart.js or D3 are completely fine to use.

Backend: Python. Use FastAPI or Flask. Keep routes, business logic and data access reasonably separated rather than piling everything into one file.

The minimum you need: an endpoint to fetch transactions, one for the coin balance, one for the rewards catalogue, and one to redeem with proper validation and error responses. Doing the pagination, filtering and sorting on the server (rather than shipping all 10k rows to the browser) is the stronger approach and something we'd love to see, but get the endpoints working first.

Database: PostgreSQL, and this one isn't optional. Use PostgreSQL 18 (the current stable release); 16 or newer is fine if your host doesn't offer 18 yet. SQLite, Mongo and in-memory stores won't be accepted. Design an actual schema rather than dumping the JSON into a single column, and include a seed script that sets up the schema and loads the provided data in one documented command.

Deploying it

Ideally you deploy both the frontend and the backend so we can open a live link, connected to a hosted Postgres. Pick whatever vendors you like; free tiers are absolutely fine (Vercel or Netlify for the frontend, Render, Railway or Fly for the backend, Neon, Supabase or Railway for Postgres are all common choices).

If time runs short and you can't get it deployed, that's okay. In that case, record a short screen video of the app running locally, walking through the main features, and put the link in your README (Google Drive, a Zoom recording, anything we can open). If you did deploy it, that video is optional and just a nice-to-have.

On AI tools

Use them. We do, every day, and we'd think it odd if you didn't. Two asks. First, drop an `AI-USAGE.md` in the repo saying which tools you used, where, and give us at least two real examples of AI output you threw away or had to fix and why. Second, know your own code cold. If you make it to the next round, you'll be walking through it and changing it live, so anything you can't explain will show.

Version control

Work in a **public GitHub repository** and commit as you go, with messages that mean something. One giant final commit with the whole project in it tells us nothing and counts against you. We do look at the history.

What to send back

Reply to this email within your 24 hours with:

- The public GitHub repo link
- Your deployed frontend and backend URLs (or the demo video link if you couldn't deploy)
- A few lines in the email itself: what's done, what isn't, and the two or three biggest assumptions you made

Inside the repo, we'll be looking for:

- A README with what the project does, the stack, local setup in under five minutes including the Postgres seed command, the live URLs, and an honest done / not-done / known-issues list
- An ASSUMPTIONS file capturing the product calls you made where the brief was vague
- A DECISIONS file for the technical choices that mattered (state management, pagination vs virtualization, schema design, and so on) with a line on why
- The AI-USAGE file described above
- The schema and seed script

Nice extras, none required: a short walkthrough video even if you did deploy, a test or two (even one meaningful test on the redeem endpoint counts), and accessibility touches like semantic markup and keyboard support.

How we'll read it

Roughly in this order of weight: your frontend engineering (React patterns, component design and reuse, TypeScript, state), your CSS and UI craft (especially that hand-built table, responsiveness, the interaction

states, overall polish), how the app copes with the full 10k rows and whatever you notice in the data, the judgement behind your assumptions and decisions, and then the backend and database fundamentals (API design, validation, schema, seed). Process counts too: commit history, a clear README, and being straight with us about what's unfinished.

Depth beats breadth here. A tight scope done really well is a stronger submission than every feature touched and none of them solid.

A couple of things that will get a submission set aside: it doesn't run following your own README, it isn't on PostgreSQL, there's neither a deployed link nor a demo video, or the repo turns out to be lifted from an existing project.

Ground rules

The 24-hour clock starts when this email lands in your inbox, and we go by whatever is pushed and live at the deadline. This assignment is only ever used to evaluate candidates; your work stays yours and never gets used commercially by us, so keep it in your public portfolio if you like. If we move forward, the next step is a technical interview where you'll walk us through your submission and extend it on the spot.

That's it. We're looking forward to seeing how you think and what you put together. Good luck.

Engineering Team, Digital Alpha Technologies